Jenkins CI/CD Pipeline Presentation Guide

Student Record Management System — Equal 4-Member Task Distribution & Live Demo Playbook

Presentation Structure Overview

This project showcases a complete Continuous Integration (CI) pipeline built with Jenkins for a Java Maven application. The presentation is divided logically into **4 equal segments** so that every member contributes clearly, demonstrating architecture, setup, pipeline mechanics, and live automated triggering/failure resilience.

Member	Presentation Role & Domain	Time Allotted	Core Deliverables
Member 1 (Pranjal)	Introduction, CI/CD Concepts & Architecture	2.5 – 3 mins	Problem statement, CI/CD benefits, architecture flow, prerequisites & Jenkins setup overview.
Member 2 (Khalid)	Jenkins Configuration, Tools & SCM Integration	2.5 – 3 mins	JDK21/Maven3 tool linking, GitHub repository connection, Git plugin setup, initial pipeline job creation.
Member 3 (Keshav)	Pipeline As Code (Jenkinsfile) & Stages	2.5 – 3 mins	Deep dive into Jenkinsfile stages: Checkout, Build, Test, Package, Archive, and JUnit reporting.
Member 4 *(Presenter 4)*	Live Demo: Triggers & Failure Demonstration	3.5 – 4 mins	Live Poll SCM automated build, intentional test failure demo, email alert verification, and restoring green state.

Detailed Member Breakdown & Speaking Script

Member 1: Pranjal — Project Context & System Architecture

Goal: Set the stage, explain why CI/CD is needed, and present the high-level architecture.

Key Points to Present:

- **What is CI/CD:** Explain Continuous Integration as an automated practice where code changes are verified through build and test runs immediately.
- **Project Overview:** Java 21 & Maven project (StudentRecordManagement) designed with unit tests, clean domain separation, and automated packaging.
- **Environment Stack:** Windows 11 host running Jenkins LTS service on port 8080, Git version control, Adoptium Eclipse JDK 21, and Apache Maven 3.9.
- **Architecture Workflow:** Developer pushes to GitHub → Jenkins detects changes → Builds & tests code → Generates artifacts & reports → Dispatches email alerts.

Speaking Tip: Use visual gestures or pointer towards the pipeline flow diagram. Keep the tone confident and introductory.

Member 2: Khalid — Tool Configuration & SCM Integration

Goal: Explain how Jenkins connects to external developer tools and GitHub.

Key Points to Present:

- **Jenkins Global Tools Setup:** Show how Manage Jenkins → Tools was configured with JDK21 and Maven3 exact path bindings.
- **Plugin Ecosystem:** Explain key plugins utilized: *Pipeline*, *Git*, *GitHub*, *JUnit*, *Mailer*, and *Timestamper*.
- **Pipeline Job Configuration:** Explain creating the StudentRecord-CI Pipeline job, using 'Pipeline script from SCM', Git repository link, and main branch pointer.
- **Security & Credentials:** Explain how public SCM requires no secrets, but SMTP notification requires secured Google App Passwords.

Speaking Tip: Emphasize that automated build execution requires exact naming match between Jenkins Tools and Jenkinsfile declarations.

Member 3: Keshav — Pipeline As Code (Jenkinsfile Anatomy)

Goal: Explain the declarative pipeline script, stages, and reporting mechanisms.

Key Points to Present:

- **Pipeline as Code Paradigm:** Explain why storing Jenkinsfile in Git is superior to GUI-only configurations (version controlled, reproducible, audited).
- **Cross-Platform Runner:** Point out the custom Groovy helper function run(cmd) executing bat on Windows or sh on Linux seamlessly.
- **Stage Breakdown:**
  1. **Checkout:** Clones repository from GitHub main branch.
  2. **Build:** Executes mvn -B clean compile to compile all Java source files.
  3. **Test:** Executes mvn -B test running JUnit 5 test suites.
  4. **Package:** Executes mvn -B package to assemble standalone snapshot JAR.
  5. **Archive Artifact:** Saves target/*.jar directly onto Jenkins dashboard with fingerprinting.
- **Post-Execution Actions:** Always records test results with junit plugin, prints status, and triggers mail on failure.

Speaking Tip: Have the Jenkinsfile on screen or printed so the audience can appreciate the clean declarative structure.

Member 4: Live Automated Trigger & Failure Demonstration

Goal: Execute the live demonstration smoothly with straightforward, pre-tested commands that never fail.

This role is the most impactful and engaging for the audience, yet simple because the exact commands are scripted step-by-step below.

Live Demo Walkthrough Steps:

Part A: Automated Trigger (Poll SCM Demo)

1. Explain that Jenkins is configured with triggers { pollSCM('H/2 * * * *') }, which checks GitHub every 2 minutes.
2. Make a trivial edit to README.md and push from terminal:

```
git commit --allow-empty -m "docs: test poll scm trigger"
git push origin main
```

3. Switch to Jenkins Dashboard and wait 1–2 minutes. Show the audience that a new build starts *by itself* without clicking any button!
4. Open the build console output and highlight line: Started by an SCM change.

Part B: Intentional Failure & Alert Demonstration (Requirement 9)

1. Tell the audience: *'Now we will prove our CI pipeline protects production by intentionally introducing a bug in our tests.'*
2. Open src/test/java/com/student/StudentServiceTest.java line 29.
3. Change assertEquals(1, service.getAllStudents().size()) to assertEquals(99, service.getAllStudents().size()).
4. Commit and push the failure:

```
git commit -am "test: intentional failure demonstration"
git push origin main
```

5. Click **Build Now** (or let Poll SCM trigger). Watch the **Test** stage fail in bright red!
6. Show the **Test Result** trend (1 failed test) and open the recipient inbox to show the instant automated failure alert email!

Part C: Restoring Green State

1. Revert line 29 back to 1 and push:

```
git commit -am "fix: restore green unit test"
git push origin main
```

2. Trigger build and show the pipeline return to 100% blue/green success.

Live Presentation Quick Reference & Cheat Sheet

Keep this cheat sheet handy during the live presentation for immediate troubleshooting and quick answers to examiner questions.

Question / Situation	Exact Response / Action
Why use Poll SCM instead of Webhooks?	Localhost Jenkins cannot receive incoming webhooks from public GitHub without a reverse proxy tunnel (e.g. ngrok). Poll SCM provides automated periodic polling without requiring public ingress.
Why did we use App Password for Gmail?	Google disabled basic authentication for security. Jenkins SMTP relies on a dedicated 16-character App Password generated under Google Account 2-Step Verification.
What happens if a stage fails?	Downstream stages (like Package and Archive) are automatically skipped. The post { always { ... } } block still publishes the JUnit test failure report, and post { failure { ... } } sends an email notification.
How does cross-platform support work?	The Jenkinsfile defines a helper run(cmd) using isUnix() check to execute bat commands on Windows and sh on Linux/Ubuntu.

Copy-Paste Terminal Commands for Live Demo:

1. Trigger Poll SCM automatically:

```
git commit --allow-empty -m "docs: test poll scm" && git push origin main
```

2. After editing StudentServiceTest.java to introduce failure:

```
git commit -am "test: deliberate failure for demo" && git push origin main
```

3. After reverting StudentServiceTest.java back to normal:

```
git commit -am "fix: restore passing tests" && git push origin main
```